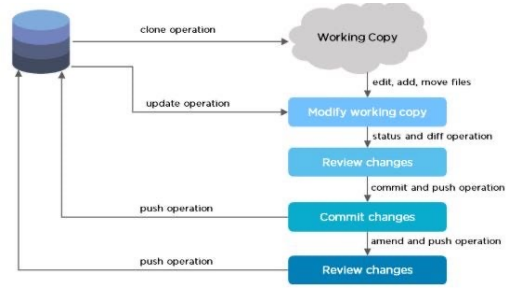
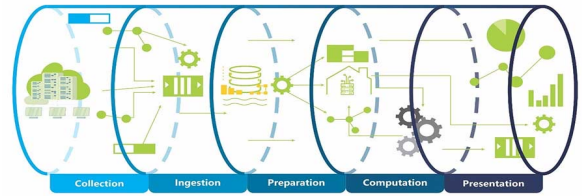
DSC 288R
Week 4 – Model Selection and Feature Engineering

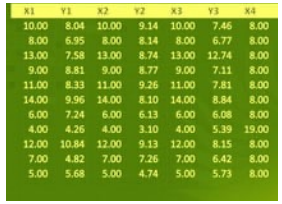
Prof. Umesh Bellur

HDSI, UCSD

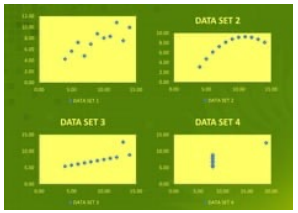
Progress so far (Week 1-3)





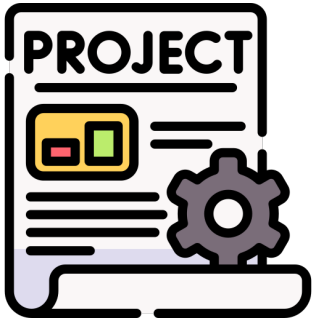


X1	Y1	X2	Y2	X3	Y3	X4
10.00	8.04	10.00	9.14	10.00	7.46	8.00
8.00	6.95	8.00	8.14	8.00	6.77	8.00
13.00	7.58	13.00	8.74	13.00	12.74	8.00
9.00	8.81	9.00	8.77	9.00	7.11	8.00
11.00	8.33	11.00	9.26	11.00	7.81	8.00
14.00	9.96	14.00	8.10	14.00	8.84	8.00
6.00	7.24	6.00	6.13	6.00	6.08	8.00
4.00	4.26	4.00	3.10	4.00	5.39	19.00
12.00	10.84	12.00	9.13	12.00	8.15	8.00
7.00	4.82	7.00	7.26	7.00	6.42	8.00
5.00	5.68	5.00	4.74	5.00	5.73	8.00



Milestones

Week	Tasks
1	Select Your Teammates
2	Submit Project Abstract
3	Create Exploratory Analysis
4	Start Modeling and Evaluation; Submit Project Progress Report
5	Continue Modeling and Evaluation
6	Continue Modeling and Evaluation
7	Prepare Project Video Presentation
8	Document Your Models
9	Submit Peer Reviews of other projects
10	Submit Final Report



Intermediate Report – Due 2/2

UC San Diego

1.Background - *Two lines on the problem you are planning to solve. Simply put, why is this problem important to be solved?*

2.Dataset name and citation/link - *Name of the dataset and a link to where its located.*

3.Data Pipeline

1. Purpose,
2. Pipeline design and details
3. Description of outputs (deduplicated data etc.)

4.EDA Description

1. Types of analysis used (Univariate, Multi-variate, Graphical/Non-Graphical)
2. Artifacts produced.

5.Feature Engineering

1. *Are you are doing feature engineering for one/ more of your planned models*
2. If Yes*:*
    1. *Features used in literature (if any?) that you are using*
    2. *Other features you intend using - Justify how you came up with these. Start from the EDA and trace a link to the feature.*

Intermediate Report (2)

UC San Diego

6. What model(s) are you using? Justify

- Name the model(s) you are using in order of highest to lowest priority for implementation

- Model Architecture from literature that you plan to use
- new architecture experiments you want to try

7.Progress Report Details on your progress.

- Effort taken on dataset collection, feature extraction, what tests you have run etc.

6.Team member contribution

- What is every member working on/planning to work on?

6.Risks and Mitigation

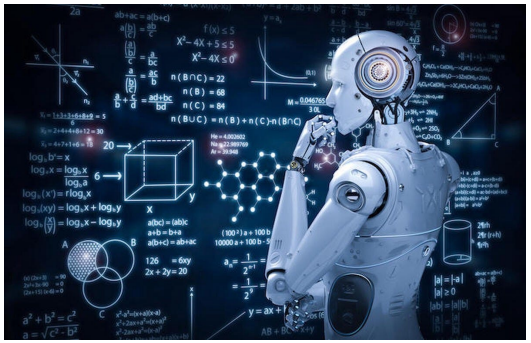
- List any risks you see at this stage to completing the project you defined in the abstract and how you are mitigating them.

6.References *- Please include relevant references.*

Model Selection

Model Selection in Data Science

- **Goal**: choose the right inductive bias for the data and task
  - Inductive bias determines what patterns a model can represent
  - Model choice impacts accuracy, robustness, and stability
  - Also affects interpretability, latency, and deployment cost
- **Focus**: principled reasoning, not model shopping



Why Model Selection Matters

- Different models fail in systematically different ways
  - Underfitting: model too simple for data structure
  - Overfitting: model memorizes noise instead of signal
  - Mismatch between data regime and model capacity
- Example: A deep NN fit training churn data well but collapsed on unseen customers.

Model Selection vs Feature Engineering

UC San Diego

- Feature engineering shapes the representation space
  - Determines which relationships are expressible
- Model selection shapes the hypothesis space
  - Determines how relationships are learned
- Example: Linear + interaction features rival tree models

Model Families and Core Assumptions

- **Linear models:**
  - Global additive effects
  - Limited interaction modeling
- **Tree models:**
  - Hierarchical, rule-based splits
- **Neural nets:**
  - Compositional feature hierarchies

Start with Baseline Models

UC San Diego

- Baselines provide a sanity check
  - Catch leakage, bugs, and label noise early
  - Establish minimum acceptable performance
  - Easier to interpret and debug

- Example: Logistic regression nearly matches XGBoost

Bias–Variance Tradeoff

- **Bias error**:
  - Model too constrained to capture structure
- **Variance error**:
  - Model overly sensitive to training data
- Model complexity controls balance
- **Example**: Polynomial regression degree tuning - Validation error clearly identified the optimal polynomial degree.

Linear Models: Strengths

- Assume linear decision boundary in feature space
  - Perform well with strong feature engineering
  - Coefficients directly interpretable
  - Fast training and inference

- Example: Credit risk models under regulation - Regulators preferred transparent coefficients over black-box gains.

Regularization in Linear Models

UC San Diego

- L1 regularization:
  - Encourages sparsity
  - Performs implicit feature selection
- L2 regularization:
  - Stabilizes correlated feature
- Example: Marketing attribution modeling - L1 revealed that only a few channels drove marketing impact.

Tree-Based Models

UC San Diego

- Automatically capture nonlinear thresholds
  - Naturally model feature interactions
  - Robust to monotonic feature transforms
- Single trees are unstable
  - Ensembles improve robustness
- Example: Fraud detection - Tree splits mirrored expert-defined fraud rules.

Bagging vs Boosting

- **Bagging**:
  - Trains models independently
  - Reduces variance via averaging
- **Boosting**:
  - Sequentially focuses on errors
  - Reduces bias

- Example: Random Forest vs XGBoost - Boosting improved recall on hard-to-detect defaulters.

GBDTs as Tabular Data Workhorse

UC San Diego

- Strong default for structured data
  - Handles mixed feature types
  - Captures complex interactions
  - Requires careful hyperparameter tuning

- Example: Kaggle tabular benchmarks - GBDTs consistently topped leaderboards on tabular benchmarks.

Kernel Methods (SVMs)

- Implicit feature expansion via kernels
  - Effective in high-dimensional spaces
  - Strong margins improve generalization
  - Training scales poorly

- Example: Small-sample text classification - SVMs excelled when labeled text data was scarce.

Distance-Based Models (k-NN)

UC San Diego

- Prediction based on similarity
  - No explicit training phase
  - Sensitive to scaling and distance metrics
  - Curse of dimensionality degrades performance

- Example: Cold-start recommendation - k-NN gave reasonable recommendations before sufficient history accumulated.

Probabilistic Models

- Explicit probabilistic assumptions
  - Model uncertainty directly
  - Often data-efficient
  - Strong independence assumptions

- Example: Naive Bayes for spam filtering - Naive Bayes remained robust despite noisy text features.

Neural Networks: Strengths

- High-capacity universal approximators
  - Learn hierarchical representations
  - Scale with data and compute
  - Require careful regularization

- Example: Image recognition with CNNs - removed the need for manual image feature design.

Deep Learning Failure Modes

- Overfitting on small or medium datasets
  - Especially severe for tabular data
- Optimization instability
  - Sensitive to initialization and learning rates
- Example: Finance datasets - Performance varied wildly across random seeds.

Model Selection for Time Series

UC San Diego

- Classical models:
  - Encode stationarity and seasonality
- ML models:
  - Depend on engineered lag features
- DL models:
  - Capture long-range dependencies
- Example: ARIMA vs LSTM - LSTM only helped once multiple correlated signals were added.

Evaluation Drives Selection

- Metrics encode business costs
  - Accuracy often misleading
- Cross-validation estimates robustness
  - Stratification for imbalance
- Example: PR-AUC vs ROC-AUC - ROC masked poor minority-class performance.

Operational and Ethical Constraints

UC San Diego

- Latency constraints limit model complexity
- Explainability may be legally required
- Fairness considerations influence model choice
- Example: Real-time credit approval - Simpler models reduced audit and compliance risk.

Deep Dive: Customer Churn

UC San Diego

- Dataset: 20k customers, 50 features
- Models evaluated:
  - Logistic regression (AUC ≈ 0.81)
  - XGBoost (AUC ≈ 0.83)
- Tradeoff:
  - Marginal accuracy vs interpretability
- Decision: Logistic regression deployed.
  - Business stakeholders trusted and adopted the simpler model faster.

Deep Dive: Fraud Detection

UC San Diego

- Dataset: 5M transactions, extreme imbalance
- Key challenges:
  - Rare positives
  - Concept drift
- Models:
  - RF vs XGBoost
- Decision: Boosted trees + PR-AUC
  - Boosted trees improved recall without overwhelming investigators.

Failure-First: When Models Break

- Deep neural networks:
  - Fail on small tabular data
- k-NN:
  - Fails in high dimensions
- SVMs:
  - Do not scale to massive datasets

- Lesson: assumptions must match data

Model Selection Decision Flowchart

- Step 1: Identify data type
  - Tabular, text, image, time series
- Step 2: Estimate data size
  - Small vs large regimes
- Step 3: Apply constraints
  - Interpretability, latency, cost

- Outcome: narrow to 2–3 candidates

Final Takeaways

- No universally best model
- Model choice is data-dependent
- Start simple and justify complexity
- Features, models, and evaluation co-evolve

- ***Successful teams revisit model choice as data evolves.***

Feature Engineering

Why Feature Engineering Dominates Model Choice

UC San Diego

Features define what information a model can access

No learning algorithm can recover missing information

Feature choice sets inductive bias implicitly

Bias–variance tradeoff is often controlled via representation

Feature Engineering in the ML Pipeline

UC San Diego

Raw data is noisy, high-entropy, and poorly structured

Logs, text, timestamps, IDs are not model-ready

Feature engineering compresses raw data into stable signals

Aggregation and normalization reduce variance

Feature Taxonomy

UC San Diego

Numerical features encode magnitudes and counts

Categorical features encode discrete states

Ordinal features encode ordered categories

Temporal features encode time dependencies

Text and unstructured features encode semantics

Numerical Features

- Numerical features represent measurable quantities or counts
  - They encode magnitude, frequency, or intensity
  - Examples: income, account balance, transaction count
- Example dataset: credit scoring
  - Raw inputs: annual income, total debt, monthly transactions

Numerical Features: Principles of Feature Selection

- Control scale so features interact properly with regularization
- Prefer normalized or ratio-based features
- Reduce skew and heavy tails using transforms
  - Ensure features are stable over time and populations

Numerical Features: Do's and Don'ts

- DO normalize or standardize features
- DO use log or power transforms for heavy-tailed variables
- DO clip extreme outliers deliberately
- DON'T use raw magnitudes blindly
  - DON'T let outliers dominate decision boundaries

Applying Do's and Don’ts: Credit Scoring

- Raw income → log(income)
  - Reduces dominance of very high earners
- Raw debt → debt-to-income ratio
  - Encodes repayment stress independent of wealth
- Raw transaction count → transactions per month
  - Removes exposure-time bias

Categorical Features

- Categorical features represent discrete, unordered states
  - No numeric distance exists between categories
  - Examples: country, subscription plan, device type
- Example dataset: customer churn prediction
  - Raw input: subscription plan = Basic / Plus / Premium

Categorical Features: Principles of Feature Selection

- Avoid imposing artificial ordering
- Control dimensionality as cardinality grows
- Preserve signal while avoiding label leakage
  - Plan for unseen categories at inference time

Categorical Features: Do's and Don'ts

UC San Diego

- DO use one-hot encoding for low cardinality
- DO use CV-safe target encoding when appropriate
- DO handle rare and unseen categories explicitly
- DON'T apply naive target encoding
  - DON'T let high-cardinality explode dimensionality

Applying Do's and Don'ts: Churn

- Subscription plan → one-hot encoding (3 plans)
  - Interpretable and low dimensional
- If plans grow → CV-safe target encoding
  - Encodes churn risk without leakage
- Unseen plan → mapped to 'other'
  - Prevents runtime failure

Ordinal Features

- Ordinal features are categorical with inherent ordering
  - Ordering matters; distances usually do not
  - Examples: credit rating, education level
- Example dataset: credit risk assessment
  - Raw input: credit rating = AAA, AA, A, BBB, ...

Ordinal Features: Principles of Feature Selection

- Preserve ordering explicitly
- Avoid encodings that destroy monotonicity
  - Check that model assumptions respect order

Ordinal Features: Do's and Don'ts

- DO encode ordinal levels as ordered integers
- DO consider monotonic constraints when available
- DON'T use one-hot encoding by default
  - DON'T assume equal spacing without justification

Applying Do's and Don'ts: Credit Rating

- AAA→1, AA→2, A→3, BBB→4, …
  - Preserves increasing risk ordering
- Tree models split naturally on thresholds
  - Linear models respect monotonic trend

Temporal Features

- Temporal features encode time-dependent structure
  - Observations are correlated across time
  - Examples: sensor readings, demand over time
- Example dataset: energy demand forecasting
  - Raw input: hourly electricity load

Temporal Features: Principles of Feature Selection

- Encode memory explicitly using lags
- Capture trends and volatility with rolling windows
- Separate calendar effects from signal
  - Respect causality strictly

Temporal Features: Do's and Don'ts

- DO use lagged and rolling features
- DO align timestamps carefully
- DO add calendar features
- DON'T use future data
  - DON'T treat time as i.i.d.

Applying Do's and Don'ts: Energy Forecasting

- Load(t−1), Load(t−24)
  - Capture short-term and daily cycles
- Rolling mean (24h)
  - Smooths transient spikes
- Hour-of-day, weekday features
  - Explain systematic demand patterns

Text Features

- Text features are unstructured and high dimensional
  - Semantic meaning is implicit, not explicit
  - Examples: reviews, support tickets, notes
- Example dataset: product review sentiment
  - Raw input: free-text customer reviews

Text Features: Principles of Feature Selection

- Reduce dimensionality while preserving meaning
- Choose representation based on task semantics
  - Balance interpretability and expressiveness

Text Features: Do's and Don'ts

UC San Diego

- DO start with TF–IDF for classical models
- DO use embeddings when semantics matter
- DO monitor vocabulary drift
- DON'T use massive raw bag-of-words
  - DON'T assume pretrained embeddings fit your domain

Applying Do's and Don'ts: Sentiment Analysis

- TF–IDF captures sentiment-laden terms
  - Down weights common neutral words
- Embeddings capture paraphrases
  - Generalize across phrasing
- Periodic retraining handles language drift
  - Maintains performance over time

Dimensionality Reduction

UC San Diego

- What it is
  - Techniques that reduce feature dimensionality while preserving signal
  - Used when features are redundant or highly correlated
- Example: numerical + interaction features in credit scoring
  - Many correlated financial ratios encode similar information
  - PCA removes correlation and stabilizes linear models
- Caution
  - Reduced features are harder to interpret and audit

Feature Leakage

- What it is
  - Using information not available at prediction time
- Example: temporal features in energy demand forecasting
  - Rolling averages accidentally include future timestamps
  - Model shows unrealistically high offline accuracy
- Why it is dangerous
  - Leaks create models that fail catastrophically in production

Missing Data as Signal

- Key idea
  - Missingness itself can carry information
- Example: credit scoring and transaction history
  - Missing income or sparse transaction data correlates with risk
  - Naive imputation destroys this signal
- Correct handling
  - Add missingness indicator features alongside imputation

Evaluating Feature Quality

UC San Diego

Why evaluation matters

Feature impact is often non-obvious

Example: credit scoring features

Ablation: remove debt-to-income ratio → performance drops

Permutation importance confirms dependency

SHAP values explain contribution direction

Goal

Detect spurious or leaked features early

UC San Diego

Thank you!